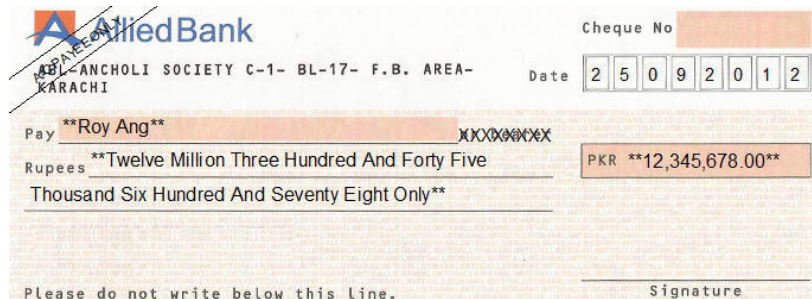


1. **Number-to-English.** If you have written a bank cheque, you may have noticed that the amount is written in words as well as in figures. This is to prevent fraud, as it is harder to change the amount in words than in figures.



Suppose you are writing a program to print a cheque. You want to convert the amount in figures to words. Write a program `num2eng.cpp` to read input an integer between $-999,999$ and $999,999$ and print the English equivalent. Here is an exhaustive list of words that your program should use: *negative, zero, one, two, three, four, five, six, seven, eight, nine, ten, eleven, twelve, thirteen, fourteen, fifteen, sixteen, seventeen, eighteen, nineteen, twenty, thirty, forty, fifty, sixty, seventy, eighty, ninety, hundred, thousand.*

Don't use hundred, when you can use thousand, e.g., use one thousand five hundred instead of fifteen hundred.

Following are some sample runs of the program.

Enter an amount in figures: 4521
In words: four thousand five hundred twenty one
Enter an amount in figures: -123
In words: negative one hundred twenty three
Enter an amount in figures: 0
In words: zero
Enter an amount in figures: 305040
In words: three hundred five thousand forty

Hint: Break the number into parts, e.g., $4521 = 4 * 1000 + 5 * 100 + 2 * 10 + 1$. Handle negative numbers by printing the word "negative" first.

2. **Fractional Arithmetic.** Write a program that implements arithmetic operations on fractions. The program should do the following:

- Input two fractions from the user (numerator and denominator for each).
- Ask the user to choose which operation to perform on the fractions (*addition*, *subtraction*, *multiplication*, or *division*).
- Display the result of the chosen operation in simplified form.

Here are sample runs of the program:

Enter numerator and denominator of first fraction: 3 4 Enter numerator and denominator of second fraction: 5 6 Choose operation (+, -, *, /): + Result: 19/12
Enter numerator and denominator of first fraction: 2 3 Enter numerator and denominator of second fraction: 4 5 Choose operation (+, -, *, /): - Result: -2/15

Hint: Use four variables to represent the two fractions (two numerators and two denominators). To add or subtract fractions, find a common denominator (the product of the two denominators is a simple choice), adjust the numerators accordingly, and then perform the addition or subtraction. To multiply fractions, multiply the numerators together and the denominators together. To divide fractions, multiply the first fraction by the reciprocal of the second fraction. Finally, simplify the resulting fraction using the GCD of the numerator and denominator.

3. **Perfect numbers.** Write a program that checks if a given number is a *perfect number*. A perfect number is a number that equals the sum of its proper divisors (excluding itself). For example, 6 is a perfect number because its proper divisors are 1, 2, and 3, and $1 + 2 + 3 = 6$.

Here are some sample runs of the program:

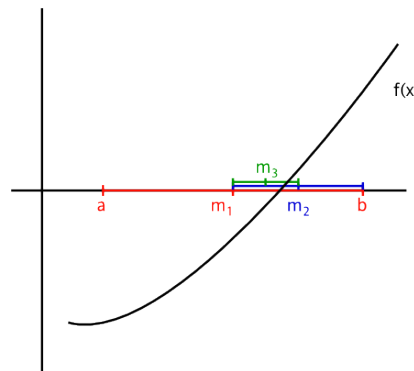
Enter a positive integer: 28 28 is a perfect number.
Enter a positive integer: 12 12 is not a perfect number.

Hint: To find the proper divisors of a number n , iterate through all integers from 1 to $n/2$ and check if they divide n evenly (i.e., $n \bmod i = 0$). If they do, add them to a sum. After checking all possible divisors, compare the sum to n to determine if it is a perfect number.

4. **Square root using bisection.** Write a program to find the square root of a positive number using the *bisection method* correct to two decimal places (without using built-in functions like `sqrt()`).

The bisection method is a numerical method for finding the root of a function. To find the square root of a number N , we can define a function $f(x) = x^2 - N$. The root of this function (i.e., the value of x for which $f(x) = 0$) is the square root of N . The bisection method works by repeatedly narrowing down an interval that contains the root. Here are the steps of the bisection method:

1. Choose an interval $[a, b]$ such that $f(a)$ and $f(b)$ have opposite signs (i.e., one is positive and the other is negative). For finding the square root of N , we can choose $a = 0$ and $b = N$ (or $b = 1$ if $N < 1$).
2. Compute the midpoint $m = (a + b)/2$ of the interval and evaluate $f(m)$.
3. If $f(m)$ is close enough to zero (within a specified tolerance), then m is an approximation of the root, and we can stop. (For two decimal places, we can use a tolerance of 0.01. That is, if $|f(m)| < 0.01$, we can stop.)
4. If $f(m)$ has the same sign as $f(a)$, then the root lies in the interval $[m, b]$, so we set $a = m$. Otherwise, the root lies in the interval $[a, m]$, so we set $b = m$.
5. Repeat steps 2-5 until the desired accuracy is achieved.



Here is how one iteration of the method works for finding the square root of $N = 25$:

- Start with the interval $[0, 25]$.
 - Compute the midpoint $m = (0+25)/2 = 12.5$ and evaluate $f(12.5) = 12.5^2 - 25 = 131.25$ (positive).
 - Since $f(0) = -25$ (negative), the root lies in the interval $[0, 12.5]$. Set $b = 12.5$.
 - Repeat the process, narrowing down the interval until the desired accuracy is achieved.
5. **Pyramid pattern.** Write a program that prints a pyramid pattern for a given number n . For example, if $n = 4$, the output should look like:

```

      1
    1 2 1
  1 2 3 2 1
1 2 3 4 3 2 1

```

Hint: Use nested loops – one for the rows and two inner loops for increasing and decreasing numbers.